

UNIT-4: Firebase and React Integration

4.1 Firebase Firestore and Realtime Database

Introduction to Firebase

Firebase is a **Backend as a Service (BaaS)** platform by Google. It provides backend features like:

- NoSQL Databases
- Authentication
- Hosting
- Cloud Functions
- Storage

Firebase removes the need to build backend from scratch for many applications.

4.1.1 Firestore vs Realtime Database

Feature	Firestore	Realtime Database
Data structure	Collection → Document	JSON Tree
Query support	Advanced	Limited
Scalability	High	Medium
Offline support	Yes	Yes
Use case	Complex apps	Simple realtime apps

4.1.2 Setting up Firebase Project and Firestore Database

Step 1: Create Firebase Project

1. Go to Firebase Console
 2. Click **Add Project**
 3. Enter project name
 4. Disable Google Analytics (optional)
 5. Create project
-

Step 2: Create Firestore Database

1. Open Firebase project
 2. Click **Firestore Database**
 3. Click **Create database**
 4. Choose **Test Mode**
 5. Select location
-

Step 3: Get Firebase Configuration

Firebase provides a config object:

```
const firebaseConfig = {  
  apiKey: "API_KEY",  
  authDomain: "PROJECT.firebaseio.com",  
  projectId: "PROJECT_ID",  
  storageBucket: "PROJECT.appspot.com",  
  messagingSenderId: "SENDER_ID",  
  appId: "APP_ID"  
};
```

This config is required in frontend apps.

4.1.3 Performing CRUD Operations on Firestore

Initialize Firestore

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";

const app = initializeApp(firebaseConfig);
const db = getFirestore(app);
```

(A) CREATE – Add Data

```
import { collection, addDoc } from "firebase/firestore";

await addDoc(collection(db, "users"), {
  name: "Om",
  email: "om@gmail.com",
  age: 21
});
```

Explanation:

- `collection()` → points to Firestore collection
 - `addDoc()` → auto generates document ID
-

(B) READ – Fetch Data

```
import { collection, getDocs } from "firebase/firestore";

const querySnapshot = await getDocs(collection(db, "users"));
querySnapshot.forEach((doc) => {
  console.log(doc.id, doc.data());
});
```

(C) UPDATE – Modify Data

```
import { doc, updateDoc } from "firebase/firestore";

await updateDoc(doc(db, "users", "docId"), {
  age: 22
});
```

(D) DELETE – Remove Data

```
import { doc, deleteDoc } from "firebase/firestore";

await deleteDoc(doc(db, "users", "docId"));
```

4.1.4 Structuring Collections and Subcollections

Firestore Structure Example

```
users (collection)
├── userId (document)
│   ├── name
│   ├── email
│   └── orders (subcollection)
│       ├── orderId
│       │   ├── product
│       │   └── price
```

Advantages

- Better scalability
- Easy querying
- Organized data

4.2 Connecting Firebase with Angular and Express

4.2.1 Integrating Firebase SDK in Angular

Install Firebase

```
npm install firebase
```

Firebase Configuration in Angular

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
```

Fetching Data in Angular Service

```
getUsers() {
  return getDocs(collection(db, 'users'));
}
```

Storing Data

```
addUser(data: any) {
  return addDoc(collection(db, 'users'), data);
}
```

4.2.2 Connecting Firebase Admin SDK in Express

Purpose

Firebase Admin SDK allows **secure backend access** to:

- Firestore
 - Authentication
 - Token verification
-

Install Admin SDK

```
npm install firebase-admin
```

Initialize Admin SDK

```
const admin = require("firebase-admin");
const serviceAccount = require("../serviceAccountKey.json");

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

const db = admin.firestore();
```

Backend API Example

```
app.get("/users", async (req, res) => {
  const snapshot = await db.collection("users").get();
  const users = snapshot.docs.map(doc => doc.data());
  res.send(users);
});
```

Advantages

- Secure access

- Server-side validation
 - No API key exposure
-

4.2.3 Firebase Hosting (Frontend Deployment)

Steps

```
npm install -g firebase-tools  
firebase login  
firebase init  
firebase deploy
```

Used For

- Angular apps
 - React apps
 - Static websites
-

4.3 Overview of React Frontend for Full Stack Developers

Why React?

- Component-based
 - Fast UI updates
 - Widely used with Firebase
-

4.3.1 Setting up React App

Using Vite

```
npm create vite@latest my-app  
cd my-app  
npm install  
npm run dev
```

Using Create React App

```
npx create-react-app my-app  
npm start
```

4.3.2 JSX, Components & Hooks

JSX

JSX allows writing HTML inside JavaScript.

```
const name = "Om";  
return <h1>Hello {name}</h1>;
```

Functional Component

```
function Welcome() {  
  return <h2>Welcome User</h2>;  
}
```

useState Hook

```
const [count, setCount] = useState(0);
```

useEffect Hook

```
useEffect(() => {  
  console.log("Component Loaded");  
});
```



```
}, []);
```

4.3.3 Integrating Firebase with React

Install Firebase

```
npm install firebase
```

Firebase Config

```
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
```

Read Firestore Data

```
useEffect(() => {
  getDocs(collection(db, "users")).then(snapshot => {
    setUsers(snapshot.docs.map(doc => doc.data()));
  });
}, []);
```

Authentication Example

```
signInWithEmailAndPassword(auth, email, password)
```

Advantages of Firebase + React

- Real-time data

- Less backend code
- Faster development
- Scalable apps